Uppsala University

Department of Materials Chemistry
Ångström Laboratory

EnGOAT

The TuTraSt Algorithm–Based Energy Geometry Analysis Toolkit

User Manual

Matevž Turk

Uppsala, 2025

Contents

1 About EnGOAT

EnGOAT (Energy GeOmetry Analysis Toolkit) is an open-source Python–Fortran package for analyzing potential energy surfaces of small species in crystalline solids. It is based on the TuTraSt algorithm [1], but extends its capabilities with additional functionality, improved performance, and a user-friendly interface. The workflow takes as input a potential energy surface of a selected probe species within a crystal lattice (specific to a given probe–host pair) and provides energetic and geometric descriptors, as well as dynamic properties of the system such as diffusion coefficients. The method is applicable to a wide range of systems, from small molecules in porous crystalline materials to solid-state ion conductors [1–3].

2 Quick Guide

This section provides a quick guide to installing and running the EnGOAT software. For a more detailed, hands-on walkthrough of the software's functionality and usage, follow the tutorial in Section 3. Furthermore, for technical details regarding the input and output file structure, or for the theoretical basis behind the EnGOAT's workflow, refer to apendicies A and B.

2.1 Installation

EnGOAT requires Python (version 3 or later) to run. Once Python is installed on your computer, installing EnGOAT is straightforward:

1. Visit `https://github.com/EnGeo-Team/EnGOAT/tree/main` and download the `EnGOAT_PYTHON_PACKAGE`.
2. Run the `install.py` script provided in the `EnGOAT_PYTHON_PACKAGE` folder. This script ensures that all required Python libraries are installed on your computer.

2.2 Running EnGOAT Analysis

The software required to perform an EnGOAT analysis is provided in the `EnGOAT_PYTHON_PACKAGE/EnGOAT` folder. To run the analysis on a `.cube` file, follow these steps:

1. Place the `.cube` file in the `EnGOAT` folder. Alternatively, you can make a copy of the folder and place the `.cube` file there, as all analysis results will be saved in the folder from which the analysis is run.
2. Modify the `input.json` file as required.
3. Run `EnGOAT.py` from the folder containing the `.cube` and `input.json` files.

2.3 Running the EnGOAT Visualization Tool

The EnGOAT visualization tool is provided in the `EnGOAT_PYTHON_PACKAGE/EnGOAT_Visualization_Tool` folder. To launch the application, simply run the `main.py` script.

Once the GUI opens, a structure can be loaded either by using the *Load* button at the bottom of the control panel or by selecting *Files/Load_structure* from the menu at the top of the control panel. This opens a dialog window in which a `.cube` file can be selected.

Important: The software does not load the `.cube` file itself. Instead, it searches for the `EnGOAT_data.json` file and the `NumPy_matrices` folder in the same directory as the selected `.cube` file. These files are generated when the EnGOAT analysis is successfully run on a `.cube` file. Therefore, it is crucial to run the EnGOAT analysis on the `.cube` file in that folder before attempting to load the structure in the visualization tool.

3 Tutorial: Li-ions in LGPS

To familiarize ourselves with the software, we will perform an EnGOAT analysis on a file named `LGPS.cube`, which contains volumetric data describing the Li-ion energy profile in *lithium germanium phosphorus sulfide* (LGPS), and then visualize the results.

3.1 Introduction

LGPS, with the chemical formula $Li_{10}GeP_2S_{12}$, is a ceramic material with exceptionally high Li-ion conductivity. For this reason, it is widely studied as a solid-state electrolyte for Li-ion batteries, where it can contribute to improved safety and higher energy density.

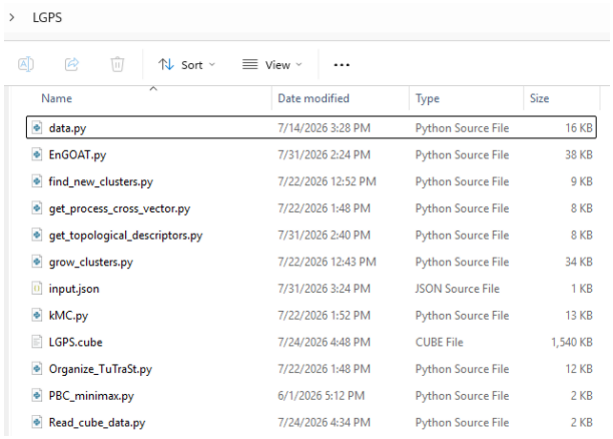
The high Li-ion conductivity of LGPS originates from its crystal structure, which enables relatively unhindered movement of Li-ions through the lattice. The mobility of Li-ions in a crystal lattice is commonly described by the *self-diffusion coefficient*, D. Consequently, D can be used as a measure of Li-ion mobility and as an indicator of the suitability of a solid-state electrolyte for Li-ion transport.

3.2 Running EnGOAT

First, we must run the EnGOAT analysis on the cube file. To do so, follow these steps:

1. Make a copy of the `EnGOAT` folder, located in the `EnGOAT_PYTHON_PACKAGE` folder, including all of its contents, and name the copy `LGPS`.

2. Copy the `LGPS.cube` file, located in `EnGOAT_PYTHON_PACKAGE/Examples`, into the newly created `LGPS` folder. The contents of the folder should look like this:



3. Open the `input.json` file. This file is used to set the parameters for the EnGOAT analysis. For this tutorial, the file should be structured as follows:

```
{
    "TuTraSt": {
        "E_unit": "kJ/mol",
        "E_step": 2,
        "E_cutoff": 60.0,
```

```
        "Dynamic_E_step": "off"
    },
    "kmc": {
        "run": true,
        "temperatures": [1000],
        "steps": 1000000,
        "runs": 5,
        "particle_mass": 6.941e-3
    },
    "cube_file": "LGPS.cube"
}
```

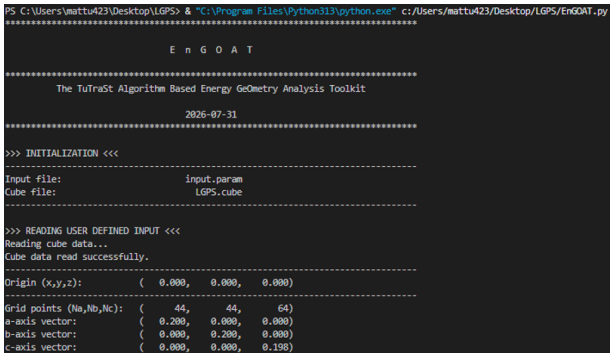
The entries in the `input.json` file control the following parameters:

- In the `TuTraSt` section, `E_unit` should be set to the energy unit in which the energy data in the cube file are expressed. `E_step` and `E_cutoff` specify the energy step and cutoff values, respectively, used by the TuTraSt algorithm and expressed in kJ/mol. `Dynamic_E_step` can be set to `"on"` or `"off"`, enabling or disabling the subroutine that dynamically adjusts the energy step of the TuTraSt algorithm based on the analyzed system.
- In the `kmc` section, the `run` parameter can be set to `true` or `false` and determines whether kMC simulations are performed after the TuTraSt analysis. `temperatures` contains a list of temperatures at which the kMC simulations are performed. The `steps` and `runs` parameters specify the number of steps in each kMC run and the number of independent runs used for averaging, respectively. `particle_mass` should be set to the mass of the diffusing particle in kg/mol.
- Finally, `cube_file` should contain the name of the `.cube` file used for the simulation. For the purposes of this tutorial, set all values in the `input.json` file as shown above.

4. Open a terminal, navigate to the `LGPS` folder, and run the following command:

```
python3 EnGOAT.py
```

Press Enter to start the analysis. The terminal should begin displaying output similar to the following:

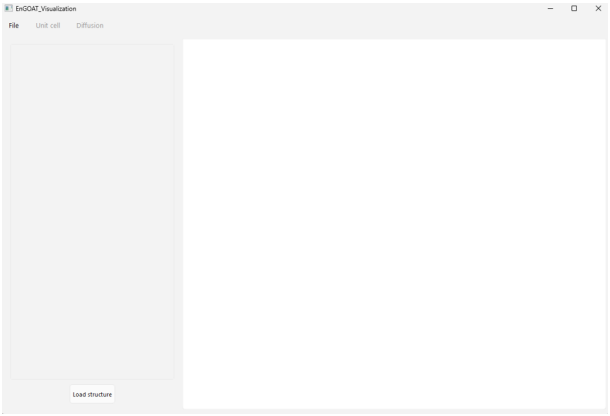


When the message `Normal termination of EnGOAT.` appears, the analysis is complete. At this point, the following files should have been generated in the `LGPS` folder:

- `output.log`
- `EnGOAT_data.json`
- A folder named `NumPy_matrices` containing five `.npy` files.

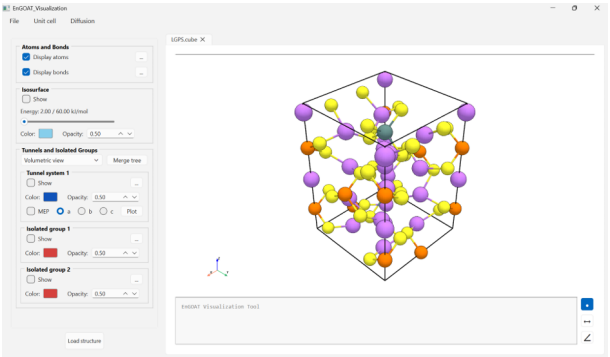
3.3 Visualizing the Results

First, launch the visualization application by opening a terminal, navigating to the `EnGOAT_PYTHON_PACKAGE/EnGOAT_Visualization_Tool` folder, and running the `main.py` script. This should open a GUI window similar to the following:



To load the LGPS structure, click the *Load structure* button at the bottom of the control panel. This will open a file selection dialog. Navigate to the `LGPS` folder and select `LGPS.cube`. The software will then begin loading the structure, which may take approximately 30 seconds to one minute. The progress of the loading process can be monitored in the terminal.

Once the loading process is complete, the structure should appear on the right-hand side of the interface, and the control panel should be updated according to the loaded structure:



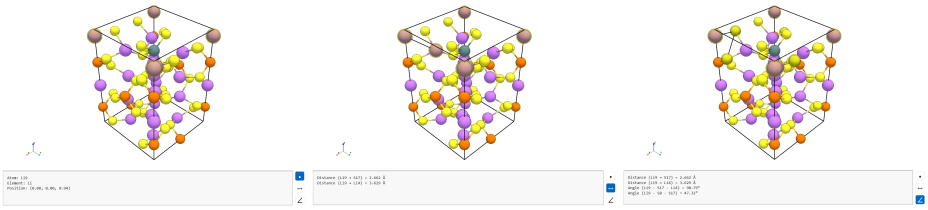
3.4 Exploring Software Functionality

3.4.1 Clicking

Upon loading, the atoms and bonds present in the unit cell of the loaded structure are displayed. All atoms are interactable, allowing the user to measure distances and angles between atoms, as well as to access information about individual atoms.

Atoms can be selected by right-clicking on them. The action performed by a right-click depends on the clicking mode selected in the bottom-right corner of the interface. The available modes are:

- **Picking mode (top button):** This is the default mode. Right-clicking an atom displays basic information about the atom, including its ID, type, and fractional coordinates.
- **Distance measuring mode (middle button):** To measure the distance between two atoms, click on the two desired atoms. Once both atoms have been selected, the PBC distance between them is displayed, and a distance vector is drawn between the two atoms.
- **Angle measuring mode (bottom button):** To measure an angle between three atoms, select all three atoms in the desired order. The second atom selected defines the apex of the angle. Once all three atoms have been selected, the angle is calculated, displayed, and drawn between the corresponding atoms.



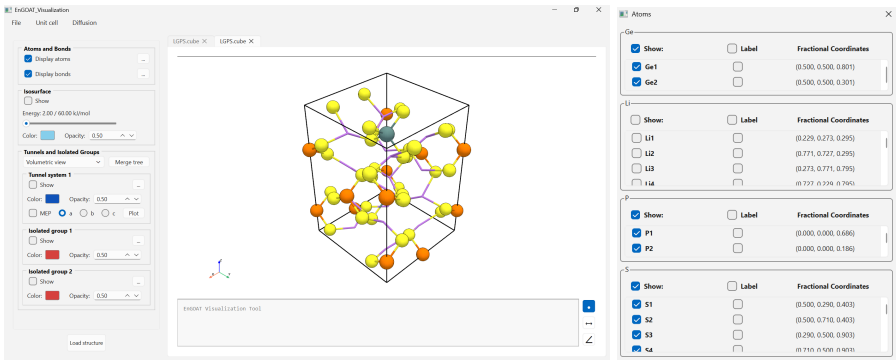
3.4.2 Displaying Atoms and Bonds

Our goal is to study the potential energy surface (PES) of Li-ions in the LGPS structure and, ultimately, derive energetic and transport properties. For this reason, we first hide the Li-ions (purple spheres) from the structure, as they are likely to be located near important minima of the Li-ion PES.

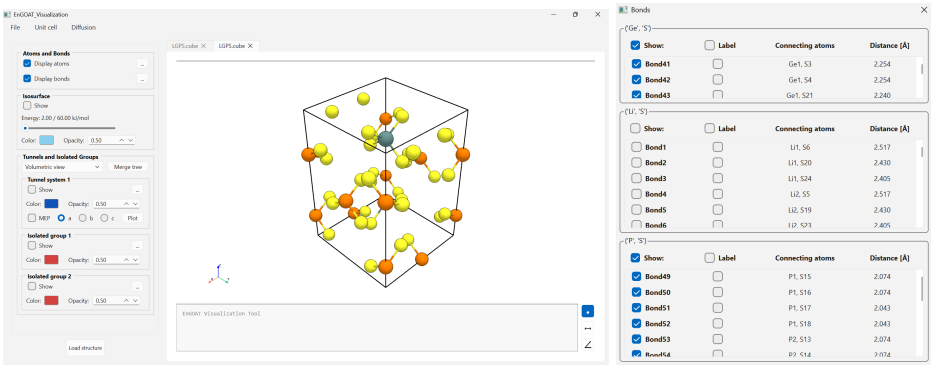
The visibility of atoms and bonds can be controlled using the *Atoms and Bonds* section located at the top of the control panel on the left-hand side of the screen.

The *Atoms and Bonds* section contains two toggles on the left, which control the visibility of all atoms and bonds, respectively. It also contains two "..." buttons on the right, which open dialogs that allow specific atoms, bonds, or atom/bond types to be displayed or hidden.

To hide the Lithium atoms, click the "..." button on the right-hand side of *Display atoms*. In the dialog window that appears, locate the *Li* section and uncheck the *Show* master toggle:

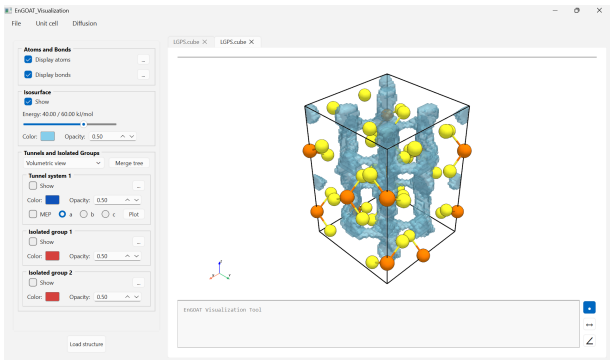


To hide all bonds, click the "…" button on the right-hand side of *Display bonds*. In the dialog window that appears, locate the *Li-S* section and uncheck the *Show* master toggle:



3.4.3 Energy Isosurface

Now that the Lithium atoms have been hidden, we can proceed by visualizing the PES of the Li-ions in LGPS. To do this, simply enable the *Show* toggle in the *Isosurface* section of the control panel and use the slider to select the desired isosurface energy level. The color and opacity of the displayed isosurface can be adjusted using the corresponding widgets in the *Isosurface* section.



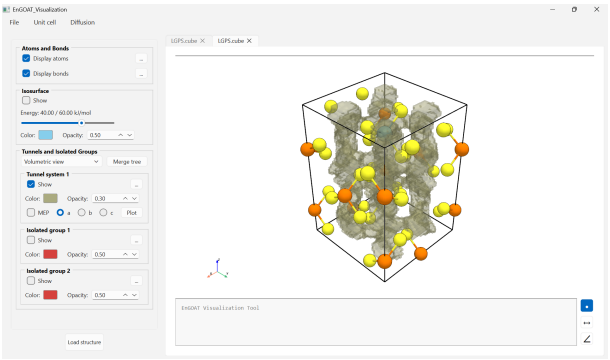
3.4.4 Tunnel Systems

The purpose of the EnGOAT analysis is to partition the PES landscape into individual *tunnel systems* and *isolated groups*. Each of these consists of *basins* (potential energy wells)

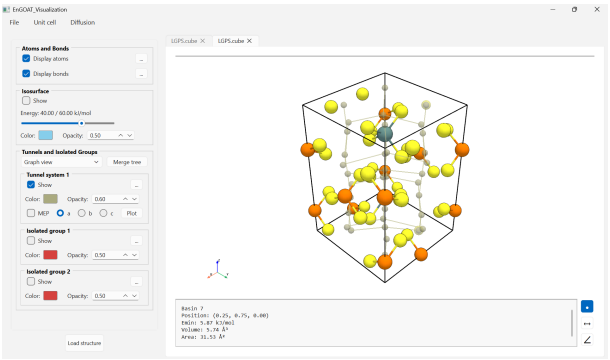
and *transition states* (surfaces connecting potential energy wells). Consequently, the central function of the visualization tool is to visualize these tunnel systems and isolated groups within the structure.

In our LGPS example, one tunnel system and two isolated groups have been identified, each with its own entry in the *Tunnels and Isolated Groups* section of the control panel. We will begin with, and primarily focus on, the visualization of the tunnel system, as it represents the most important part of the PES for our purposes.

To visualize the tunnel system, first disable the *Show* toggle in the *Isosurface* section so that the isosurface does not obscure the tunnel system. Then, enable the *Show* toggle for *Tunnel system 1*. This will display the percolating-channel-forming part of the PES. The color and opacity of the displayed tunnel system can be adjusted using the corresponding widgets in the *Tunnel system 1* section.



As mentioned above, the tunnel system is composed of individual *basins* and *transition states*, which together form a periodic network in which the basins act as graph nodes and the transition states represent connections between them. To visualize the tunnel system in this form, switch from *Volumetric view*, which displays the entire volume of the individual basins and transition states, to *Graph view* using the dropdown menu located above the *Tunnels and Isolated Groups* section.



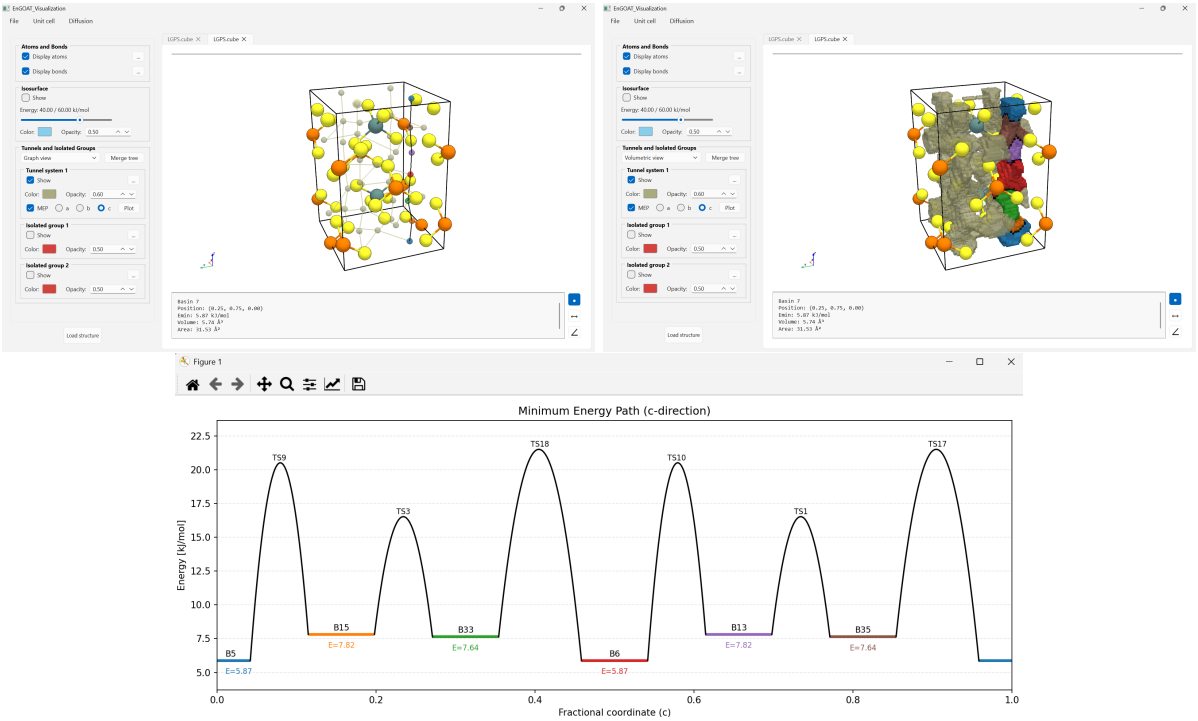
In *Graph view*, the individual basins and their connectivity become immediately apparent. Similar to atoms, basins in *Graph view* are clickable objects, allowing distances and angles to be measured and individual basins to be explored.

One of the most important metrics characterizing a tunnel system is the *minimum energy pathway* (MEP) along a given crystallographic direction. To visualize an MEP, enable the

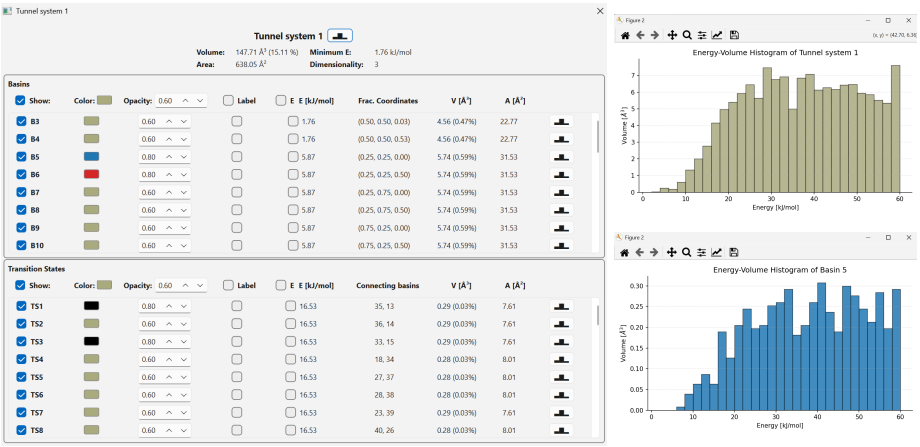
MEP toggle and select the desired crystallographic direction. This will highlight the basins and transition states that constitute the selected MEP.

Furthermore, an energy diagram of the MEP can be plotted by clicking the *Plot* button on the right-hand side of the MEP selection row. The resulting plot shows the relevant basins positioned according to their corresponding fractional coordinates on the x-axis and their birth energies on the y-axis. The basins are displayed using the same colors as in the visualization window, while the transition states and the corresponding energy barriers between the basins are also shown.

To visualize the full volume of the MEP, switch back to *Volumetric view*. The individual basins and transition states will remain highlighted and distinguishable.

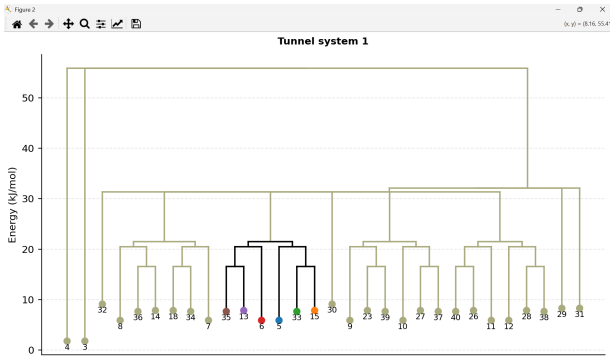


To explore the basins and transition states of the tunnel system in more detail, click the "..." button in the upper-right corner of the *Tunnel system 1* section. This opens a dialog containing detailed information about the tunnel system, including all of its constituent basins and transition states. The appearance of individual basins and transition states, as well as the display of their labels and birth energies, can be modified in this dialog. Energy-volume histograms for the entire tunnel system and for individual basins and transition states can also be plotted using the corresponding histogram buttons.



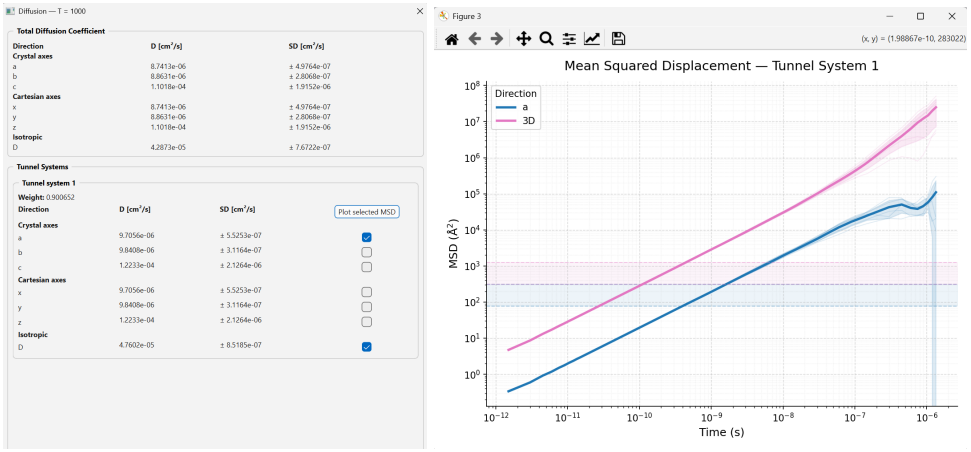
Finally, the connectivity of the tunnel system can be visualized using a merge tree diagram. To do this, first ensure that all tunnel systems and isolated groups that should be included in the diagram are displayed by enabling their respective *Show* toggles. Then, click the *Merge tree* button located in the upper-right corner of the *Tunnels and Isolated Groups* section of the control panel.

This will open a merge tree diagram in which the basins are plotted at their corresponding birth energies on the y-axis and connected through the relevant transition states. The elements of the diagram are color-coded consistently with their appearance in the interactive visualization window.



3.4.5 Diffusion

Since we have chosen to run the kMC simulations on the graph network after the TuTraSt analysis (by setting the ”run” parameter in the ”kMC” block to True), the Li-ion transport property data at every specified temperature is available. It can be accessed by clicking on the ”Diffusion” button in the overhead menu at the top edge of the screen, and selecting the temperature. This opens a dialog window with the overall and per-tunnel-system self diffusion coefficients along all crystallographic as well as cartesian directions. For each tunnel system, the MSD curves from which the self-diffusion coefficients were calculated can also be plotted by selecting the desired directions and clicking the ”Plot MSD” button.



3.5 Exercises

The tutorial above provided a comprehensive overview of the software and its functionality. Below are several open-ended questions that can be answered by exploring the LGPS structure and applying the knowledge gained from the tutorial to the EnGOAT package.

- ***Li-ion Coordination Environments***: In LGPS, there are several potential energy basins for Li-ions; however, they are not all energetically equivalent.

    1. Find one example of a basin at each unique minimum energy value. Use the dialogs in the *Tunnel system* and *Isolated groups* sections to obtain a comprehensive overview of the available basins.
    2. Using the interactive clicking functionality, determine the coordination number of each selected basin and measure the distances to its neighbouring sulfur atoms. Use this information to explain the differences in the birth energies of the basins. To simplify the analysis, unwanted atoms and bonds can be hidden using the dialogs in the *Atoms and Bonds* section.

- ***Anisotropic Li-ion Diffusion***: LGPS is known to be a Li-ion superconductor; however, its Li-ion conductivity is not equivalent in all crystallographic directions.

    1. Using the *Diffusion* menu, determine along which crystallographic direction Li-ion diffusion is fastest. Verify your findings by plotting and comparing the MSD curves along all crystallographic directions.
    2. Explore the MEPs along the different crystallographic directions and examine the corresponding potential energy plots. Relate the observed differences in diffusion rates between the crystallographic directions to the structural and energetic properties of LGPS revealed by the MEPs.

4 References

(1) Mace, A.; Barthel, S.; Smit, B. Automated multiscale approach to predict self-diffusion from a potential energy field. *Journal of chemical theory and computation* **2019**, *15*, 2127–2141.

(2) Gustafsson, H.; Kozdra, M.; Smit, B.; Barthel, S.; Mace, A. Predicting ion diffusion from the shape of potential energy landscapes. *Journal of Chemical Theory and Computation* **2023**, *20*, 18–29.

(3) Schwarz, F.; Barthel, S.; Mace, A. Understanding Mobile Particles in Solid-State Materials: From the Perspective of Potential Energy Surfaces. *Chemistry of Materials* **2024**, *36*, 11359–11376.

(4) Gustafsson, H.; Schwarz, F.; Smolders, T.; Barthel, S.; Mace, A. Computationally efficient DFT-based sampling of ion diffusion in crystalline solids. *Journal of Chemical Theory and Computation* **2025**, *21*, 8669–8682.

(5) Frenkel, D.; Smit, B., *Understanding molecular simulation: from algorithms to applications*; Elsevier: 2023.

(6) Bennett, C. H. Exact defect calculations in model substances. *Diffusion in Solids: Recent Developments.,* **1975**, 73–113.

(7) Chandler, D. Statistical mechanics of isomerization dynamics in liquids and the transition state approximation. *The Journal of Chemical Physics* **1978**, *68*, 2959–2970.

(8) Gillespie, D. T. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of computational physics* **1976**, *22*, 403–434.

A Technical Details

A.1 Input Files

EnGOAT requires two input files:

- *Input parameter file*: This file contains all the user-defined inputs required to run the code. It must always be named `input.json`.
- *Potential energy grid file*: This file contains the potential energy surface of the chosen probe within the selected crystal lattice, represented on a grid. The file uses the `.cube` format.

Both files must be placed in the directory in which `EnGOAT.py` is run. The structure of both files is discussed in detail below. The example input files presented in this section relate to the Li-ions in LGPS example of the tutorial.

A.1.1 Input Parameter File

An example of the input parameter file `input.json` is shown in Listing 1. As noted above, this file *must* be named `input.json` for the software to recognize it.

Listing 1: The input parameter file `input.param`.

```
{
    "TuTraSt": {
        "E_unit": "kJ/mol",
        "E_step": 2,
        "E_cutoff": 60.0,
        "Dynamic_E_step": "off"
    },
    "kmc": {
        "run": true,
        "temperatures": [1000],
        "steps": 1000000,
        "runs": 5,
        "particle_mass": 6.941e-3
    },
    "cube_file": "LGPS.cube"
}
```

The entries in the `input.json` file control the following parameters:

- *`TuTraSt` section*:
    1. *`E_unit`*: sets the energy unit in which the volumetric data of the `.cube` file is stored. Choose between: `kJ/mol`, `kcal/mol`, `Ry`, `eV`, `Hartree`.
    2. *`E_step`*: sets the energy step *(in kJ/mol)* used during the TuTraSt flooding algorithm (see Appendix B).
    3. *`E_cutoff`*: sets the energy cutoff value *(in kJ/mol)* for the TuTraSt flooding algorithm.

4. *Dynamic_E_step*: dinamically adjusts the `E_step` value based on the first breakthrough value of the structure (see Appendix B). Can be turned `on` and `off`.

- *kMC section*:

  1. *run*: dictates whether the kMC simulations for the computation of the self-diffusion coefficient are performed. Can be set to `true` or `false`.
  2. *temperatures*: specifies temperatures at which the kMC simulations are to be run. The temperatures must be given in a python list format as `[T1, T2, ...]`
  3. *steps*: sets the number of steps performed per kMC simulation run.
  4. *runs*: sets the number of independent kMC runs to be performed for averaging.
  5. *particle_mass*: sets to the mass of the diffusing particle in kg/mol.

- *cube_file*: contains the name of the `.cube` file used for the simulation.

A.1.2 Cube File

An example of the cube file storing the potential energy grid data is shown in Listing 2. The name of this file is arbitrary, but it must be specified in the input parameter file (see Section A.1.1) for the software to recognize it. The file is structured as follows:

- *Lines 1–2:* Comment lines of the cube file. These lines are ignored when reading the data.
- *Line 3:* The first integer specifies the number of atoms in the repeating unit. The following three values give the Cartesian coordinates of the origin of the volumetric data.
- *Lines 4–6:* The first integer on each line specifies the number of voxels (grid points) along each lattice direction. The following three values define the Cartesian components of the voxel basis vectors corresponding to the lattice vectors $\vec{a}$, $\vec{b}$, and $\vec{c}$, respectively. It should be noted that the Cartesian spatial coordinates in `.cube` files are given in units of *bohr*; the conversion to *Ångström* is performed internally by the software. If the coordinates are instead provided in *Ångström*, the resulting analysis will be incorrect.
- *Lines 7–13:* These lines specify the atom information: the atomic number (first value), the atomic charge (second value), and the Cartesian coordinates of each atom within the repeating unit (third, fourth, and fifth values, respectively). The number of lines must exactly match the number of atoms specified in *Line 3* to ensure correct program operation.
- *Lines 14–end:* The volumetric data encoding the potential energy grid. The data start at the origin and proceed through all values along the $\vec{a}$ direction, gradually filling the ab plane, and then filling successive layers along the $\vec{c}$ direction. The energy unit in which the volumetric data is given is arbitrary, but must be specified in the input parameter file (see Section A.1.1).

Listing 2: The `.cube` file storing potential energy grid data of Li-ions in LGPS.

```
cube file - LGPS
--------------------------------
   50     0.000000     0.000000     0.000000
   44     0.377414     0.000000     0.000000
   44     0.000000     0.377414     0.000000
   64     0.000000     0.000000     0.373739
    3     0.000000     3.797815     4.532674     7.045739
    3     0.000000    12.808428    12.073569     7.045739
    3     0.000000     4.532674    12.808428    19.005386
...(44 more atoms)
   16     0.000000     8.303122     4.859352    16.699638
   16     0.000000    11.746891     8.303122     4.739991
   16     0.000000     4.859352     8.303122     4.739991
89.946964
214.611112
214.611112
214.611112
... (more volumetric data)
```

A.2 Output Files

EnGOAT generates the following output files during execution:

- `output.log`: the log file containing the details of the run.
- `EnGOAT_data.json`: A json file containing all the data produced during the run in a compact python dictionary.
- `NumPy_matrices`: matrices storing the voxel energies and positions of individual basins and transition states on the grid.

The structures of the output files are described in detail below. The example output files presented in this section were generated during the Li-ions in LGPS example of the tutorial.

A.2.1 `output.log`

The `output.log` file records the progression of the simulation together with the key metrics evaluated during runtime. It is organized into multiple sections, each corresponding to a distinct stage of the EnGOAT analysis workflow as follows:

1. *Initialization*: This section displays the names of the input files from which the simulation parameters and PES grid data were read.
2. *Reading User-Defined Input*: This section shows important parameters related to the repeating unit encoded in the `.cube` file, such as the number of atoms in the repeating unit, the origin of the Cartesian coordinate system, the number of grid points (voxels) along each unit cell vector direction, and the Cartesian coordinates of the unit cell vectors in Ångstrom.

3. *Potential Energy Grid Analysis*: This section shows the progression of the flooding algorithm used to partition the PES grid into individual *basins* and *transition state surfaces* (see Appendix B). At the beginning, the number of energy levels, as well as the user-defined values for the energy step and the cutoff energy (in kJ/mol), are displayed. This is followed by a table summarizing the analysis at each energy level. The columns, in order, are: the energy level number; the corresponding energy value; the energy volume, i.e., the number of grid points at or below the current energy level; the unit cell vector direction of all breakthroughs recorded up to the current level ("/" if no breakthroughs were recorded, or [a,b,c] if a breakthrough has been recorded, where a, b, and c are 1 if the repeating unit was breached in the given unit cell vector direction and 0 otherwise); the time spent analyzing the current energy level; and the total runtime.

   If the *dynamic energy step correction* option is selected in the input parameter file (see Section A.1.1), the energy levels are evaluated until the first breakthrough is recorded. If this occurs between levels 10 and 20, the analysis proceeds normally; otherwise, it is restarted with a modified value of the energy step.

4. *TuTraSt Analysis (lines 48–74):* This section presents the geometric descriptors resulting from the TuTraSt analysis of the PES grid, in which individual basins and transition states are grouped into *tunnel systems* (see Appendix B). First, the *total* and *accessible* volume and surface area of the entire system are reported. The total volume and surface area are obtained by summing over all basins in the system, whereas the accessible quantities are summed only over basins that belong to tunnel systems (excluding isolated groups). The volume is reported in $Å^3$ and as a percentage of the total unit cell volume, while the surface area is reported in $Å^2$, as well as normalized by the total unit cell volume.

   Subsequently, metrics describing each individual tunnel system are provided. In addition to the volume and surface area of a given tunnel system, the lowest-energy point within the tunnel system and the dimensionality of the tunnel system (defined as the number of linearly independent directions of percolating channels forming the tunnel system) are reported. Finally, a table listing all directions of percolating channels that constitute the tunnel system and their corresponding minimal highest energy barriers is provided for each tunnel system. Each direction is given by a vector [a,b,c], where a, b, and c correspond to the number of unit cells traversed along the respective crystallographic directions. The minimal highest energy barrier that a particle must overcome to traverse a repeating unit along a given direction is obtained by applying the minimax algorithm, under periodic boundary conditions, to a graph representation of the tunnel system (see Appendix B).

5. *Running kMC Simulations*: This section presents the results of the kMC simulations performed for each tunnel system at each temperature specified by the user. At the beginning, the user-defined number of kMC runs and the number of kMC steps per simulation are reported. This is followed by one table for each tunnel system.

   At the top of each table, the tunnel system identifier (ID) and its weight (the tunnel system's contribution to the total diffusion of the system, calculated from the Boltzmann

probability of finding a particle in a given tunnel system (see Appendix B)) are reported. The table then lists the self-diffusion coefficients D_a, D_b, and D_c along the unit cell vector directions a, b, and c, respectively, together with the time spent on each run (Δt) and the cumulative runtime of the kMC simulations.

At the bottom of the table, the averaged self-diffusion coefficients and their corresponding standard deviations for the given tunnel system are reported along the a, b, and c directions, as well as along the Cartesian x, y, and z directions, together with the isotropic diffusion coefficient.

After all tunnel systems have been analyzed, the joint (weight-averaged) diffusion coefficients for the entire system at the specified temperature are reported.

6. *Summary*: This section presents a concise summary of the run. First, the geometric descriptors (volume and surface area) are listed, followed by a table showcasing the energy levels at which breakthroughs in each unit cell vector direction were first recorded, and the minimal highest energy barriers for traversing along these directions. At the end of the section is a table listing the self-diffusion coefficients along the crystallographic directions at all specified temperatures.

A.2.2 `EnGOAT_data.json`

The `EnGOAT_data.json` file contains all of the data produced during an EnGOAT run. The data are organized and stored as a single Python dictionary within the JSON file. The dictionary contains the following keys:

- `metadata`: contains a dictionary with the input parameters, such as the energy step and cutoff, as well as information about the computational grid.
- `unit_cell`: contains geometric and energetic descriptors of the entire system, including the volume, surface area, and energy histogram.
- `atoms`: contains information about all atoms in the structure, including their types and Cartesian coordinates.
- `basin_data`: contains information about all basins in the structure, including their positions in grid coordinates and their energetic and geometric descriptors.
- `TS_data`: contains information about all transition states in the structure.
- `tunnel_systems`: contains information about all tunnel systems in the structure, including their graph-network representations and minimum energy pathways (MEPs).
- `isolated_groups`: contains information about all isolated groups in the structure.
- `kMC_data`: contains information about the self-diffusion coefficients along all crystallographic and Cartesian directions, as well as the isotropic self-diffusion coefficient. It also contains the MSD curves from all kMC runs across all directions and all identified

tunnel systems. If kMC simulations were not performed, the `kMC_data` entry in the dictionary has the value *None*.

An example of the `EnGOAT_data.json` file is shown in Listing 3.

Listing 3: The `EnGOAT_data.json` file.

```
{
    "metadata": {
        "E_step": 2,
        "E_cutoff": 60.0,
        "N_levels": 30,
        "origin": [0.0, 0.0, 0.0],
        "grid_shape": [44, 44, 64],
        "grid_vectors": [
            [0.199718902254086, 0.0, 0.0],
            [0.0, 0.199718902254086, 0.0],
            [, 0.0, 0.19777417586401097],
        ],
        "grid_spacing": [0.199718902254086,  0.199718902254086, 0.19777417586401097],
        "V_voxel": 0.00788874511185925,
        "energy_unit": "kJ/mol",
        "temperature_list": [1000]
    },
    "unit_cell": {
        "A_tot": 674.7148212054536,
        "A_acc": 638.0543948259968,
        "V_tot": 155.58183109608825,
        "V_acc": 147.7088634744527,
        "V_UC": 977.4470743398085,
        "V_B_tot": {
            "1000": 888.9173910616034
        },
        "histogram": {
            "0": 0.11044243156602951,
            "2": 0.410214745816681,
            ...
        }
    },
    "atoms": {
        "Li1": {
            "Z": 3,
            "type": "Li",
            "center": [2.009717293910935, 2.398587957933826, 3.7284447811920107]
        },
        ...
    },
    "basin_data": {
        "1": {
            "center": [0, 0, 28],
            "E_min": 0.0,
            "V": 3.9364838108177658,
```

```
            "A": 18.330213189728358,
            "V_rel": 0.004027311466942149,
            "A_rel": 0.018753151624203313,
            "histogram": {
                "2.0": 0.03944372555929625,
                "4.0": 0.0788874511185925,
                ...
            },
            "V_B": {
                "1000": 44.156177936884674
            },
            "group_type": "isolated_group",
            "group_ID": 1
        },
        ...
    }
    "TS_data": {
        "1": {
            "basins": [35, 13],
            "start_center": [12, 11, 52],
            "end_center": [10, 11, 42],
            "cross_vector": [0, 0, 0],
            "E_min": 16.52803,
            "V": 0.29188356913879226,
            "A": 7.612595827037192,
            "V_rel": 0.000298618285123967,
            "A_rel": 0.007788243503801906,
            "histogram": {
                "2.0": 0.0,
                "4.0": 0.0,
                ...
            },
            "V_B": {
                "1000": 1.5178371426630877
            },
            "group_type": "tunnel_system",
            "group_ID": 1
        },
        ...
    },
    "tunnel_systems": {
        "1": {
            "basin_list": [
                3,
                ...
            ],
            "TS_list": [
                1,
                ...
            ],
            "E_min": 1.756088,
```

```
            "dimensionality": 3,
            "V": 147.7088634744527,
            "A": 638.0543948259968,
            "V_rel": 0.151116993801653,
            "A_rel": 0.6527764127351389,
            "histogram": {
                "2.0": 0.031554980447437,
                "4.0": 0.252439843579496,
                ...
            },
            "V_B": {
                "1000": 800.6050351878345
            },
            "graph_network": {
                "3": [
                    {
                        "end_basin": "5",
                        "TS": "41",
                        "cross_vector": [0, 0, 0]
                    },
                    ...
                ]
            }
            "MEPs": {
                "a": {
                    "path": [
                        {
                            "start_basin": 29,
                            "end_basin": 35,
                            "transition_state": 33,
                            "crossing": 0
                        },
                        ...
                    ]
                },
                "b": {...},
                "c": {...}
            }
        },
        ...
    },
    "isolated_groups": {
        "1": {
            "basin_list": [
                1
            ],
            "TS_list": [],
            "E_min": 0.0,
            "V": 3.9364838108177658,
            "A": 18.330213189728358,
            "V_rel": 0.004027311466942149,
```

```
            "A_rel": 0.018753151624203313,
            "histogram": {
                "2.0": 0.03944372555929625,
                "4.0": 0.0788874511185925,
                ...
            },
            "V_B": {
                "1000": 44.156177936884674
            }
        },
        ...
    },
    "kMC_data": {
        "1000": {
            "D_tot": {
                "a": {
                    "D": 8.741336343689696e-06,
                    "sd": 4.976360373392653e-07
                },
                "b": {
                    "D": 8.863136809921906e-06,
                    "sd": 2.8067886528610593e-07
                },
                "c": {
                    "D": 0.00011017522526285008,
                    "sd": 1.9151701032024637e-06
                },
                "x": {
                    "D": 8.741336343689696e-06,
                    "sd": 4.976360373392653e-07
                },
                "y": {
                    "D": 8.863136809921906e-06,
                    "sd": 2.8067886528610593e-07
                },
                "z": {
                    "D": 0.00011017522526285008,
                    "sd": 1.9151701032024637e-06
                },
                "3D": {
                    "D": 4.287313719775982e-05,
                    "sd": 7.672188850614935e-07
                }
            },
            "tunnel_systems": {
                "1": {
                    "weight": 0.900651785236983,
                    "directions": {
                        "a": {
                            "D": 9.705567109257039e-06,
                            "sd": 5.525287858151809e-07,
```

```
                                "MSD": {
                                    "run_1": {
                                        "time": [
                                            1.5008278546898717e-12,
                                            3.001156991367343e-12,
                                            ...
                                        ],
                                        "MSD": [
                                            0.3369321487316422,
                                            0.5900220216149196,
                                            ...
                                        ]
                                    },
                                    ...
                                }
                            },
                            ...
                        }
                    }
                    ...
                }
            }
        }
}
```


A.2.3 NumPy_matrices

The complete 3-dimensional matrices containing the locations of individual basins, transition states, and tunnel systems are stored in `.npy` format in the directory `NumPy_matrices`. This directory has the following structure:

```
numpy matrices/
|-- Basin_matrix.npy
|-- Energy_matrix.npy
|-- Level_matrix.npy
|-- TS_matrix.npy
|-- Tunnel_matrix.npy
```

- `Basin_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given basin contain the ID of that basin, while points not belonging to any basin are assigned a value of 0. Points that were identified as transition states are assigned the value of -1.
- `Energy_matrix.npy`: A numpy matrix with the same number of points as the PES grid. It contains the volumetric data from the `.cube` file, with the lowest energy shifted to 0 and all values above the cutoff set to the cutoff value.

- `Level_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Each point holds the value of the energy level at which it was identified. Points in the PES grid with energy values above the energy cutoff are assigned the value *number of levels + 1*.
- `TS_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given transition state contain the ID of that transition state, while points not belonging to any transition state are assigned a value of 0.
- `Tunnel_matrix.npy`: A numpy matrix with the same number of points as the PES grid. Points belonging to a given tunnel system contain the ID of that tunnel system, while points not belonging to any tunnel system are assigned a value of 0.

B The EnGOAT Workflow

In this section, the EnGOAT workflow [1] is explained in detail and illustrated using a simple example: a system containing a single diffusing particle moving through a two-dimensional, cell-centered tetragonal crystal lattice. The particle can occupy vacant basins located at the centers of clusters of four atoms, and can move between them via transition states situated between the two atoms that connect neighboring basins, as shown in Figure 1 (left). Such a crystal lattice can be described by a repeating unit, highlighted in Figure 1 (right). It is important to note that the choice of the repeating unit used in the EnGOAT workflow is not always straightforward and depends on the system in question [4], a point that will be further discussed in Section B.1.

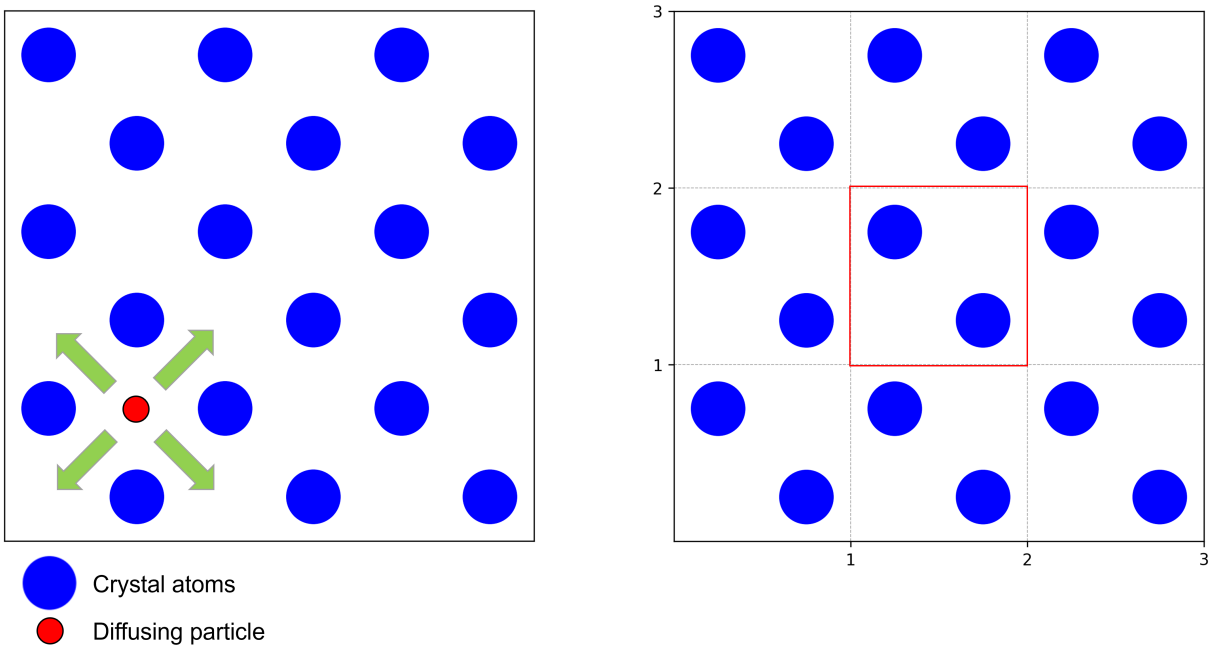


Figure 1: A two-dimensional system consisting of a diffusing particle in a cell-centered tetragonal crystal lattice (left), and the chosen repeating unit used to describe the lattice (outlined in red, right).

In general, the EnGOAT workflow can be split into three parts:

1. *Construction of the potential energy grid:* A potential energy surface (PES) experienced by the diffusing particle is generated on a grid within the chosen repeating unit of the crystal structure. This part of the workflow is not implemented in our software and must be performed using external tools (such as RASPA).

2. *TuTraSt analysis:* The PES is analyzed using the TuTraSt flooding algorithm to identify individual basins accessible to diffusing particles and the transition states between them. From this analysis, several geometric descriptors representative of the system are computed.

3. *Kinetic Monte Carlo (kMC) simulation:* Based on the TuTraSt analysis, a graph network

of basins and possible transitions between them is constructed, and the self-diffusion coefficients of the diffusing particle along the crystal lattice axes are calculated via kMC simulations.

Each of these steps is discussed in detail in the following sections of this chapter.

B.1 Construction of the Potential Energy Grid

The first part of the EnGOAT workflow involves constructing the potential energy surface (PES) experienced by the diffusing particle within the crystal lattice. As noted in the previous section, this part of the workflow is not implemented in our software and must be carried out using external tools, such as RASPA. The procedure for calculating such grids consists of the following steps:

1. *Choosing the repeating unit:* While in some cases simply choosing the unit cell as the repeating unit is sufficient, this is not always true, and careful consideration is required. The choice depends on the nature of the system and the method used for energy calculations. For certain systems—particularly charged systems where quantum chemistry methods such as DFT are employed—if the repeating unit is too small, the calculated energies may depend on its size [4]. In such cases, a sufficiently large supercell must be chosen to eliminate size-dependent effects in the energy calculations.

2. *Constructing a grid on the chosen repeating unit:* Once an appropriate repeating unit is chosen, a three-dimensional grid is overlaid on the structure along the crystal lattice vector directions, serving as a framework for single-point energy calculations. There are several considerations regarding the grid size: a grid that is too coarse may fail to capture the details of the potential energy surface, while a grid that is too fine can lead to high computational cost. It has been found that a voxel size of approximately 0.15–0.2 Å (length of each side) is optimal, although in some cases a size of up to 0.4 Å can be used if appropriate interpolation techniques are applied [4].

3. *Calculating the single particle PES:* Once the grid is constructed, a probe representing the diffusing particle is placed at the center of each voxel, and a single-point energy calculation is performed to determine the energy at that position in the crystal lattice. This procedure is repeated for every voxel in the grid, resulting in the single-particle potential energy surface (PES). The procedure is illustrated in Figure 2. The choice of method for the single-point energy calculations should also depend on the system. For neutral, weakly interacting probes, such as an H_2 molecule, classical force field calculations are usually sufficient. However, more challenging probes—particularly charged ions—require calculations at a higher level of theory, such as density functional theory (DFT) or wavefunction-based quantum methods.

For neutral, weakly interacting probes, single-particle PES provide a sufficient description of the potential energy experienced by particles in a given crystal lattice, as interactions between the diffusing particles are short-ranged. In the case of charged ions, however, the electrostatic

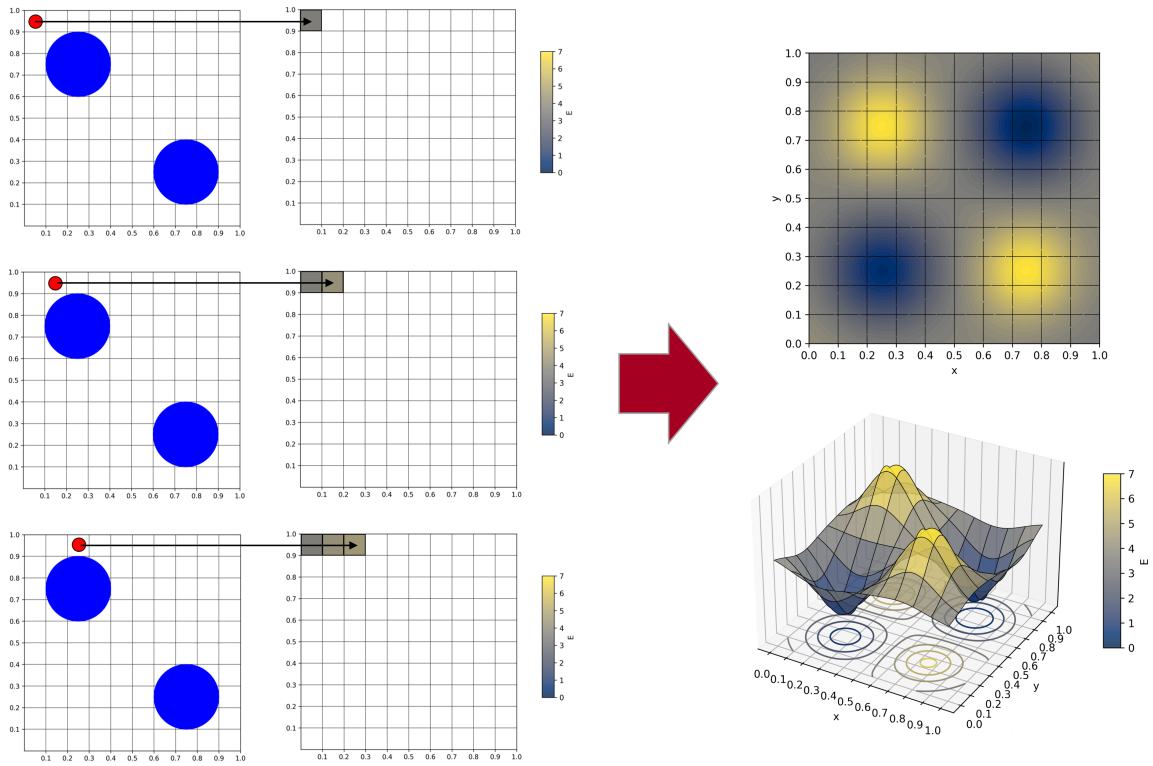


Figure 2: Construction of the single particle potential energy grid.

interactions between particles are long-ranged and influence the effective potential within the lattice. Consequently, single-particle PES are an inadequate description of the system and must be corrected. One approach is the so-called ionic TuTraSt [2], in which a single-particle PES is first constructed, and then a Monte Carlo (MC) simulation is performed on the PES grid populated with the desired number of ions to represent the target loading. From the MC simulation, the probability density of finding ions at various points in space is extracted, and this information is used to calculate an energy correction, which is added to the single-particle PES grid. The result is an effective multi-particle PES grid that accounts for inter-particle interactions.

B.2 TuTraSt analysis

In the second step of the EnGOAT workflow, the PES grid (constructed as described in Section B.1) is partitioned into separate *basins* and *transition states* using the TuTraSt flooding algorithm. Based on this partitioning, a structural analysis of the system is subsequently performed, yielding geometric descriptors of the system.

Before the start of the analysis, the energy levels are discretised, transforming the PES grid into an *energy level matrix*, i.e., a 3D grid in which each voxel contains the energy value at that point in space, floored to the nearest discrete energy level. This process is illustrated in Figure 3. The step between energy levels is a user-defined input. The appropriate value of the energy step varies between systems and has been found to be roughly an order of magnitude smaller than the energy of the first *breakthrough* (i.e., the energy value at which the first percolating channel through the crystal lattice is recorded, see below). Our software also provides an option to automatically and dynamically adjust the energy step to satisfy

this criterion. Additionally, an energy cutoff value is applied to the energy level matrix (also defined by the user): all voxels with an energy level at or above the cutoff value are considered inaccessible (i.e., their energy is set to infinity).

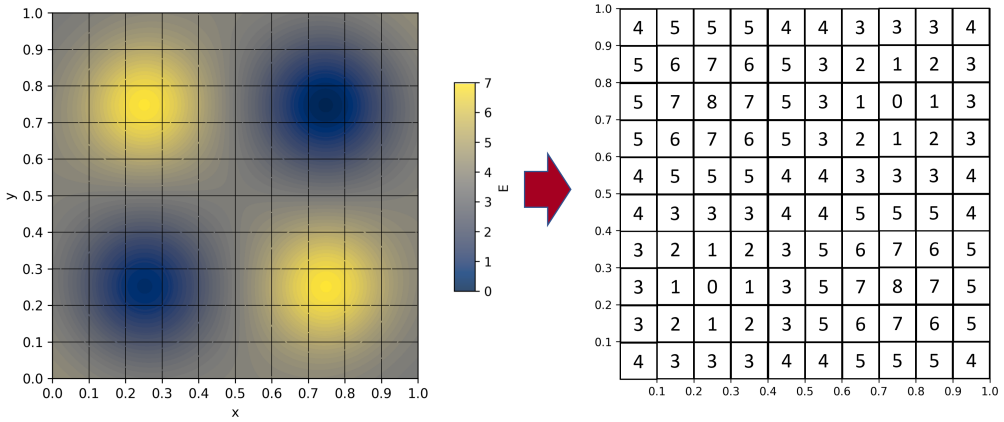


Figure 3: Construction of the energy level matrix. In this case, the energy step is equal to 1 energy unit.

Following the construction of the energy level matrix, the flooding algorithm is applied to partition the space within the PES into individual *basins*—i.e., potential energy wells—and to identify *transition state points*, i.e., points where two adjacent basins merge. The algorithm proceeds by analyzing points in the energy level matrix at each energy level in turn, starting at level 0 and increasing by the energy step until reaching the energy cutoff. At each energy level, the following steps are executed in order:

1. *Initiating new basins:* The energy level matrix is checked for points (or groups of points) at the current energy levels that have no neighbours that are already asingned to a basin (all the neighbours are at a higher energy level). If any such points are found, a new basin is initiated at each such position.

2. *Growing existing basins and identifying transition state points:* After the initiation of new basins, all existing basins are expanded at the current energy level, one layer of points at a time, using the neighbourhood search algorithm. At each basin's border, neighbouring points that do not yet belong to any basin are assigned to the current basin. If a neighbouring point already belongs to a different basin, it is removed from both basin point lists and designated as a transition state point between the two basins. Once basins are connected by at least one transition state point, they are merged into a *basin cluster*, together with all basins connected to the merged basins. If a basin cluster forms a percolating channel through the crystal structure (i.e., forms a continuous path through the repeating unit) in any direction, a *breakthrough* is recorded along that direction at the given energy level. This process is repeated until all points at the current energy level have been processed.

The progression of the flooding algorithm is illustrated in Figure 4.

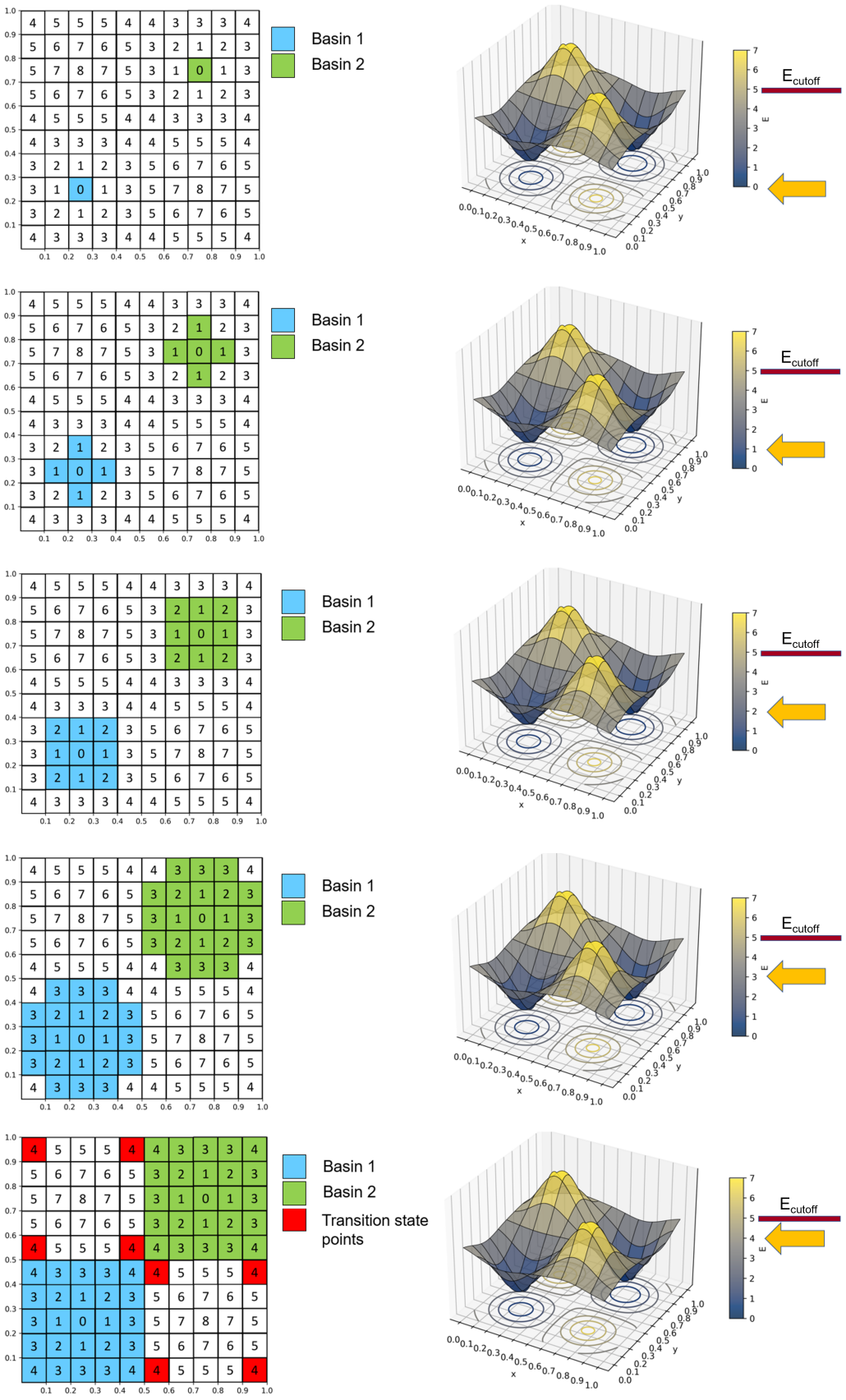


Figure 4: Progression of the flooding algorithm. In this case, the cutoff energy value is equal to 5 energy units.

Once all points at energies below the energy cutoff value have either been assigned to basins or identified as transition state points, the TuTraSt (Tunnel–Transition state) analysis is performed. The first step involves the organisation of *transition states*: all adjacent transition state points between a given pair of basins are merged to form a single transition state corresponding to the transition of a diffusing particle between those two basins.

In the second step, basin clusters within which at least one breakthrough was recorded are identified as *tunnel systems*. Each tunnel system is characterized by its *dimensionality*—the number of linearly independent *directions* of percolating channels within the tunnel system. Each direction is assigned a *breakthrough energy* (the energy at which the breakthrough was first recorded) and a *lowest energy barrier*—the smallest possible maximum energy that a particle must overcome to traverse the tunnel system in that direction. The lowest energy barrier is determined by representing the tunnel system as a graph network, where basins are nodes connected by transition edges with associated energy barriers, and applying a modified minimax algorithm with applied periodic boundary conditions to the graph. Basin clusters in which no breakthroughs were recorded are labeled as *not tunnel-forming*, and are labeled as *Isolated groups*. The tunnel system characterization process is illustrated in Figure 5.

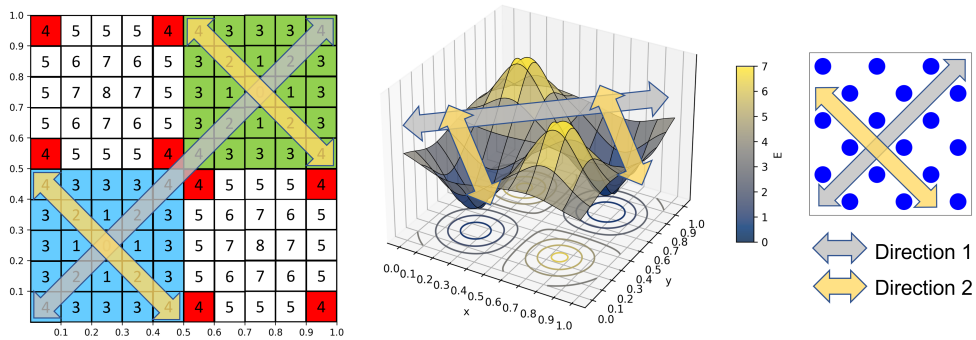


Figure 5: Tunnel system characterization. This example shows a two-dimensional tunnel system, consisting of two basins connected by four transition states, with two breakthrough directions. In both cases, the breakthrough energy and lowest energy barrier are equal to 4 energy units.

Furthermore, additional geometric descriptors, such as the surface area S and volume V, can be readily extracted from the partitioned energy level matrix. The surface area is calculated by summing the areas of all voxel faces that border inaccessible points (i.e., points with energies higher than the cutoff value), as given by Equation 1:

$$S = \sum_{\text{border faces}} \left[(\vec{a} \times \vec{b})\, \delta_{\vec{c}} + (\vec{a} \times \vec{c})\, \delta_{\vec{b}} + (\vec{b} \times \vec{c})\, \delta_{\vec{a}} \right], \tag{1}$$

where $\vec{a}$, $\vec{b}$, and $\vec{c}$ are the side vectors of the voxels forming the grid, and $\delta_{\vec{i}}$ is the *Kronecker delta*, which equals 1 if the face is normal to $\vec{i}$ and 0 otherwise. The volume can be calculated

by summing the volumes of all accessible voxels, as given by Equation 2:

$$V = \sum_{\text{accessible voxels}} \left| \vec{a} \cdot (\vec{b} \times \vec{c}) \right| \tag{2}$$

Both surface area and volume can be reported in the following contexts:

- *Total area/volume:* sum over *all* points with energies below the cutoff value.
- *Accessible area/volume:* sum over points that belong to tunnel systems; points that belong to isolated groups (not tunnel-forming basin clusters) are excluded from this metric.
- *Area/volume per tunnel system:* sum only over points belonging to an individual tunnel system.

The surface area and volume calculations are illustrated in Figure 6.

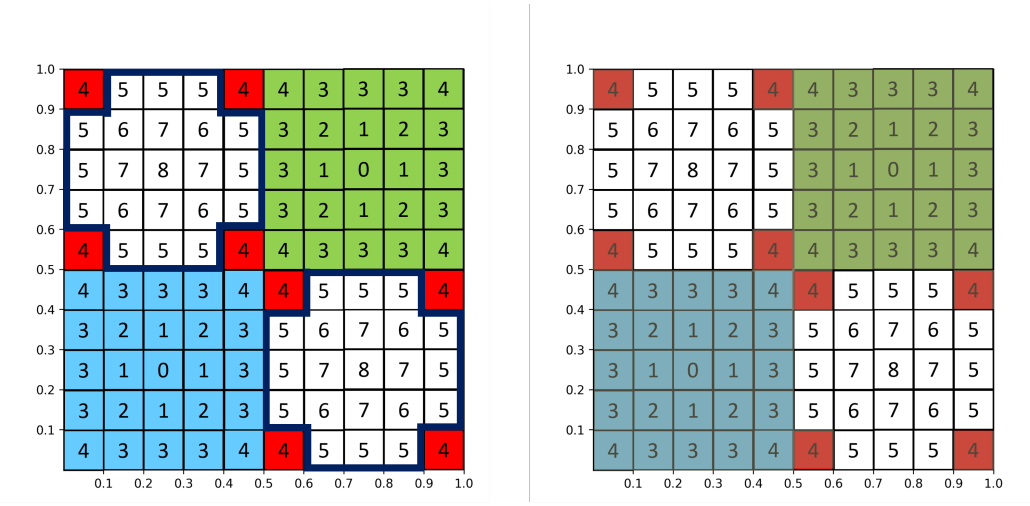


Figure 6: Surface area (dark blue outline, left) and volume (shaded region, right) of the system. In this two-dimensional example, the surface area is equal to 4 units of length, and the volume is equal to 0.58 units of area. The total and accessible values of S and V are identical, as both basins are included in the tunnel system.

B.3 Kinetic Monte Carlo Simulations

The third step of the EnGOAT workflow involves the calculation of self-diffusion coefficients along the crystal lattice axes using kinetic Monte Carlo (kMC) simulations.

First, each tunnel system identified during the TuTraSt analysis (Section B.2) is converted into a three-dimensional graph network. In this representation, individual basins within the tunnel system are treated as nodes, positioned at the spatial coordinates of the lowest-energy point of each basin. The nodes are connected by edges representing *transition processes*.

Each transition process is characterized by a *transition process vector* $\vec{v}$, defined as the vector from the center of the initial basin (basin 1) to the center of the final basin (basin 2), which encodes the displacement length and direction of the diffusing particle, and by a *transition*

process rate k, which quantifies the probability that the process occurs per unit time. The transition process rate for a particle migrating from basin 1 to basin 2 via a given transition state is obtained using the Bennett–Chandler approach [5–7] (Equation 3):

$$k_{1\to 2} = \kappa \sqrt{\frac{k_B T}{2\pi m}}\, P(q^*), \tag{3}$$

where κ is the transmission (symmetry) factor (for an ideal transition state, $\kappa = \frac{1}{2}$), T is the temperature, m is the particle mass, and $P(q^*)$ is the probability of finding the particle at the transition-state coordinates q^*, i.e., at the top of the energy barrier separating basins 1 and 2. This probability is calculated by taking the Boltzmann-weighted sum over all points in the transition state region and normalizing it by the sum over basin 1 and the transition state region (Equation 4):

$$P(q^*) = \frac{\int_{\mathrm{TS}} e^{-E(q^*)/k_B T}\, dq^*}{\int_{\mathrm{basin\ 1}} e^{-E(q)/k_B T}\, dq + \int_{\mathrm{TS}} e^{-E(q^*)/k_B T}\, dq^*}. \tag{4}$$

For each pair of basins connected by a unique transition state, two transition processes are defined (two per transition state if multiple transition states connect the same basin pair): one from basin 1 to basin 2, with vector $\vec{v}$ and rate $k_{1\to 2}$, and one in the reverse direction, with vector $-\vec{v}$ and rate $k_{2\to 1}$. The construction of the portion of the graph network corresponding to two basins connected via a transition state is illustrated in Figure 7.

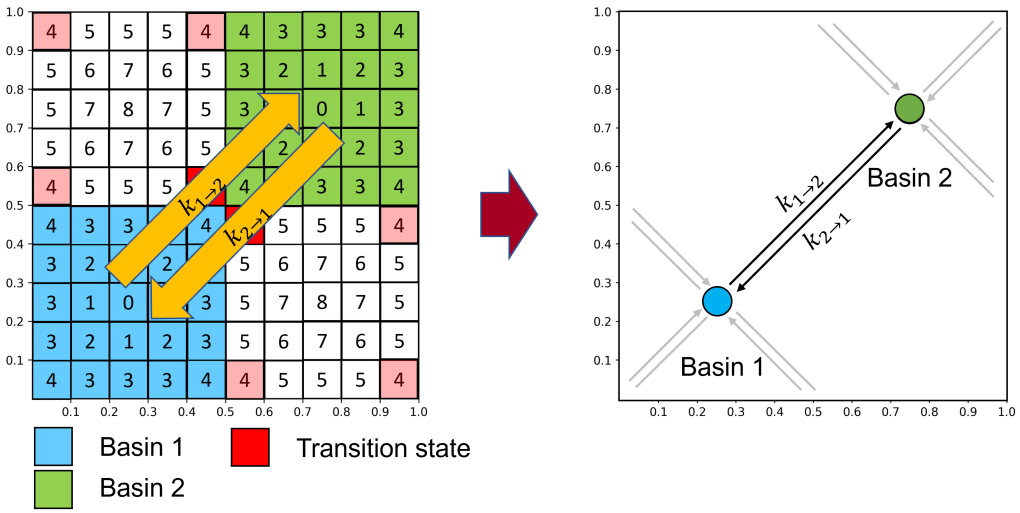


Figure 7: Construction of the portion of the graph network corresponding to basins 1 and 2 connected via the highlighted transition state. The transition process vectors are obtained by taking the difference between the coordinates of the basin centers, and the transition process rates are calculated using Equation 3. To construct the full graph network for this example, the same procedure must be applied to the other three transition states connecting basins 1 and 2.

Once the full graph network for each tunnel system identified within the crystal lattice is constructed, a set of kMC simulations is performed for each tunnel system in order to determine its contribution to the self-diffusion of the diffusing particle along the individual crystal lattice axes. The self-diffusion coefficient is determined using the following procedure:

1. The diffusing particle is placed in a randomly selected basin (i.e., a random node in the tunnel-system graph). Time is set to $t = 0$, and the particle's initial position is set to $\vec{r} = (0, 0, 0)$.

2. From all available transition processes originating in the basin currently occupied by the particle (i.e., all outgoing edges from the current node), one process is selected at random. The selection probability is weighted by the corresponding transition process rate k_i and normalized by the sum of all outgoing rates, as given by Equation 5:

$$P_i = \frac{k_i}{\sum_{\text{basin}} k_i}, \tag{5}$$

   where P_i is the probability of selecting the i-th process and the denominator represents the sum over all transition process rates starting from the current basin. After selecting the process, the particle is moved from the initial basin to the target basin. The simulation time is advanced according to the Gillespie algorithm [8] (Equation 6):

$$\Delta t = -\frac{\ln(\rho)}{\sum_{\text{basin } I} k_i}, \tag{6}$$

   where ρ is a random number uniformly distributed between 0 and 1. The particle position is updated by adding the displacement vector of the chosen transition process. This step is repeated N times (where N is a user-defined parameter), yielding a particle trajectory $\vec{r}(t)$, as illustrated in Figure 8.

3. From the particle trajectory, mean square displacement (MSD) curves are computed along the crystal lattice directions:

$$\text{MSD}_i = \left\langle \left(r_i(t) - r_i(t_0)\right)^2 \right\rangle, \tag{7}$$

   where MSD_i is the mean square displacement along direction i, and $r_i(t)$ and $r_i(t_0)$ are the particle coordinates along that direction at the time t and the chosen initial time t_0, respectively.

   The diffusion coefficient along direction i in a tunnel system T is obtained from the slope of the MSD curve:

$$D_i^T = \frac{1}{2} \lim_{t\to\infty} \frac{d}{dt} \left\langle \left(r_i(t) - r_i(t_0)\right)^2 \right\rangle. \tag{8}$$

   To ensure that the diffusive regime is reached, the slope is evaluated only over the portion of the trajectory where the particle has traveled between one and two unit-cell lengths along the respective direction. If the particle fails to traverse at least two unit-cell lengths within the simulation time, the diffusion in that direction is considered negligible and the corresponding diffusion coefficient is set to zero.

For each tunnel system, this procedure is repeated multiple times, and the diffusion coefficient is taken as the average over all runs.

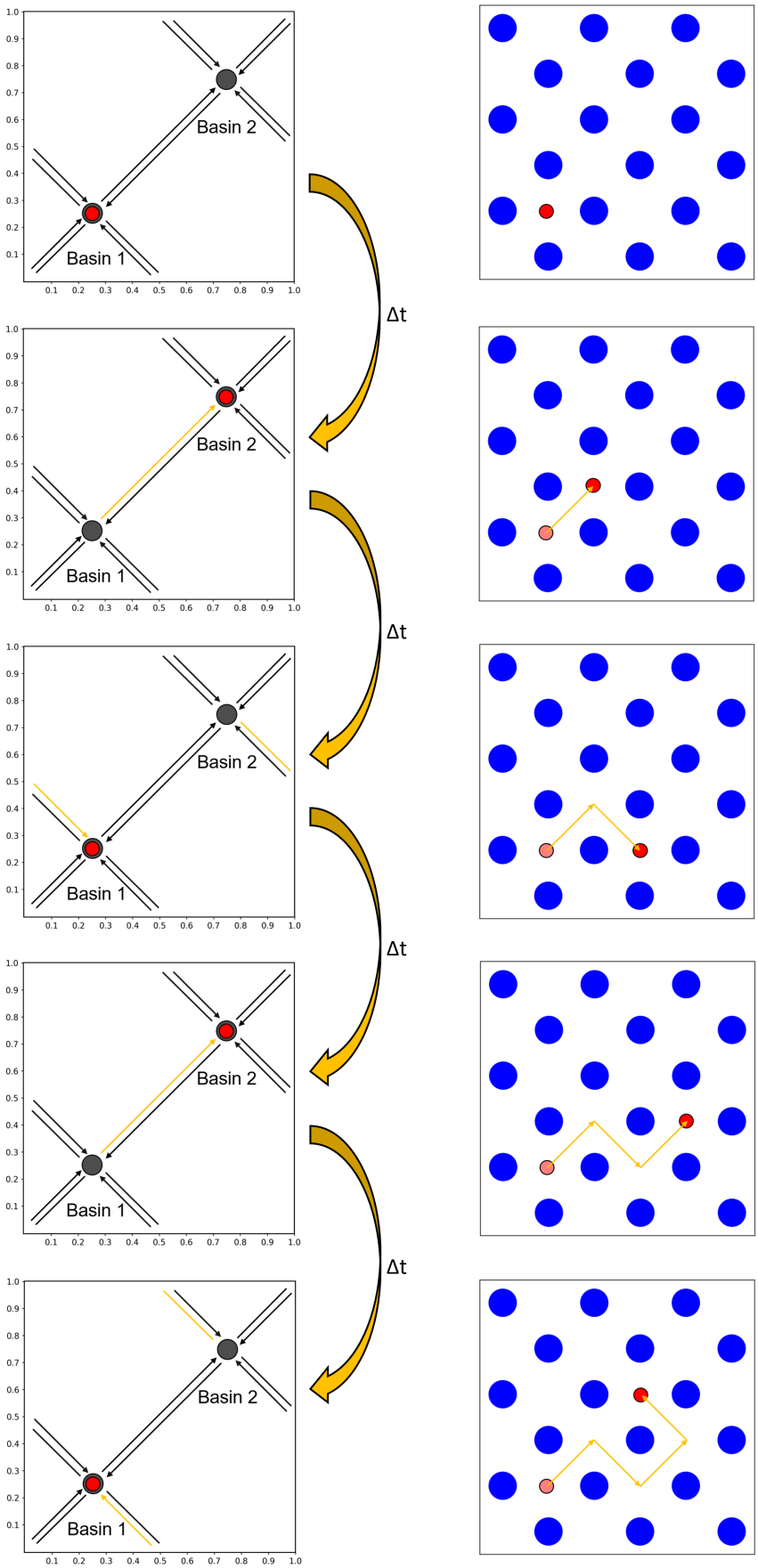


Figure 8: Progression of the kMC simulation. The right-hand side of the figure shows the trajectory of the diffusing particle, $\vec{r}(t)$.

To obtain the total diffusion coefficients along the crystal lattice direction i, a weighted average over all tunnel systems is computed:

$$D_i = \sum_T P(T)\, D_i^T, \tag{9}$$

where the weight $P(T)$ is the probability of finding the particle within tunnel system T. It is calculated as the Boltzmann-weighted sum over all voxels belonging to that tunnel system, normalized by the Boltzmann-weighted sum over the total accessible volume of the system (i.e., all voxels with energy below the cutoff):

$$P(T) = \frac{\int_T e^{-E(q)/k_B T}\, dq}{\int_V e^{-E(q)/k_B T}\, dq}. \tag{10}$$